

Contents

- [Generic questions](#)
- [Vendasta-specific questions](#)

Caching Strategy

Caching stores the result of an expensive read or computation in a faster, closer store, so later requests are served from it instead of redoing the work. Done right it transforms latency and cost. Done wrong it introduces stale data and consistency bugs. The hard part is invalidation, not the cache itself.

Generic questions

1. What is caching?

Caching stores the result of an expensive read or computation in a **faster, closer store**, so later requests can be served from it instead of redoing the work.

2. Why do we use caching?

To serve data faster and to take load off the source. A cache hit skips a slow database, network call, or computation.

3. What problems does caching solve?

Slow repeated reads, expensive recomputation of the same result, and downstream systems overloaded by duplicate traffic.

4. What are the benefits of caching?

- **Latency:** reads come from memory, not disk or network.
- **Throughput:** fewer calls to the source, so it serves more unique work.
- **Cost:** fewer paid API calls and compute cycles.

5. What are the drawbacks of caching?

- **Staleness:** the cache can lag the source.
- **Complexity:** invalidation and consistency are hard to get right.
- **Another failure mode:** the cache itself can go down or fill up.
- **Memory cost:** caches hold data in expensive RAM.

6. When should you avoid caching, and is it always beneficial?

No, it is not always beneficial. Avoid it when data changes faster than it is read, when reads are rare (low hit ratio), when the source is already cheap, or when the data must be strongly consistent, like account balances or live inventory.

7. What makes a good candidate for caching?

Data that is **read far more than it is written**, expensive to produce, and can tolerate small staleness: product catalogs, user profiles, config, computed aggregates.

8. What types of data should never be cached?

Anything that must be exact at read time (account balances, live stock), one-time secrets or tokens you must not reuse, and highly sensitive per-user data unless the cache is properly scoped and secured.

9. Explain caching with a real-world example.

A library keeps popular books on a front shelf instead of walking to the back archive each time. The shelf is the cache: fast for popular books, refreshed when a book changes.

10. Cold cache v/s hot cache.

A **cold cache** is empty or just started, so most requests miss and hit the source (slow). A **hot cache** holds the working set, so most requests hit (fast). **Warming** loads the cache before traffic to skip the cold phase.

11. What are the main types of cache, and which fits which scenario?

Cache	What it holds and where	Use when
Client-side	Data on the client device	The client re-reads the same data
Browser	HTTP responses and assets, via Cache-Control	Static assets, repeat visits
CDN	Assets and cacheable responses at edge nodes	Global static or media content
DNS	Domain-to-IP mappings, in resolver and OS	Avoiding repeat DNS lookups
Reverse proxy	HTTP responses in front of the app (Varnish, nginx)	Shared response cache for many users
API gateway	Responses at the gateway	Idempotent GET endpoints
Application (in-process)	Objects in the app's own memory (Caffeine)	Very hot per-instance data
In-memory	Data in RAM, local or in Redis	General fast lookups
Distributed	Shared cache across instances (Redis, Memcached)	Shared state, scale-out
Database	Buffer pool and query cache	Repeated queries and hot pages
Operating system	Page cache for files and disk	Repeated file reads
CPU	L1/L2/L3 for hot memory	Hardware, automatic
Object	Cached deserialized objects	Avoiding recompute or re-deserialization
Query	Cached query results	Repeated identical queries
Page	Cached rendered pages or fragments	Full-page or fragment rendering
Session	User session data (Redis session store)	Session lookups across instances

Rule of thumb: caches closer to the user are faster but harder to invalidate; caches closer to the data are slower but easier to keep correct.

12. Why use Redis, and what is its architecture?

Redis is an in-memory data-structure store: strings, hashes, lists, sets, sorted sets, and streams, with TTLs, pub/sub, and scripting. **Architecture:** a mostly single-threaded event loop (so commands are atomic), optional persistence (RDB snapshots and the AOF log), primary/replica replication, and Redis Cluster for sharding. Use it when you need rich data types, persistence, or more than a plain key-value cache.

13. Why use Memcached, and what is its architecture?

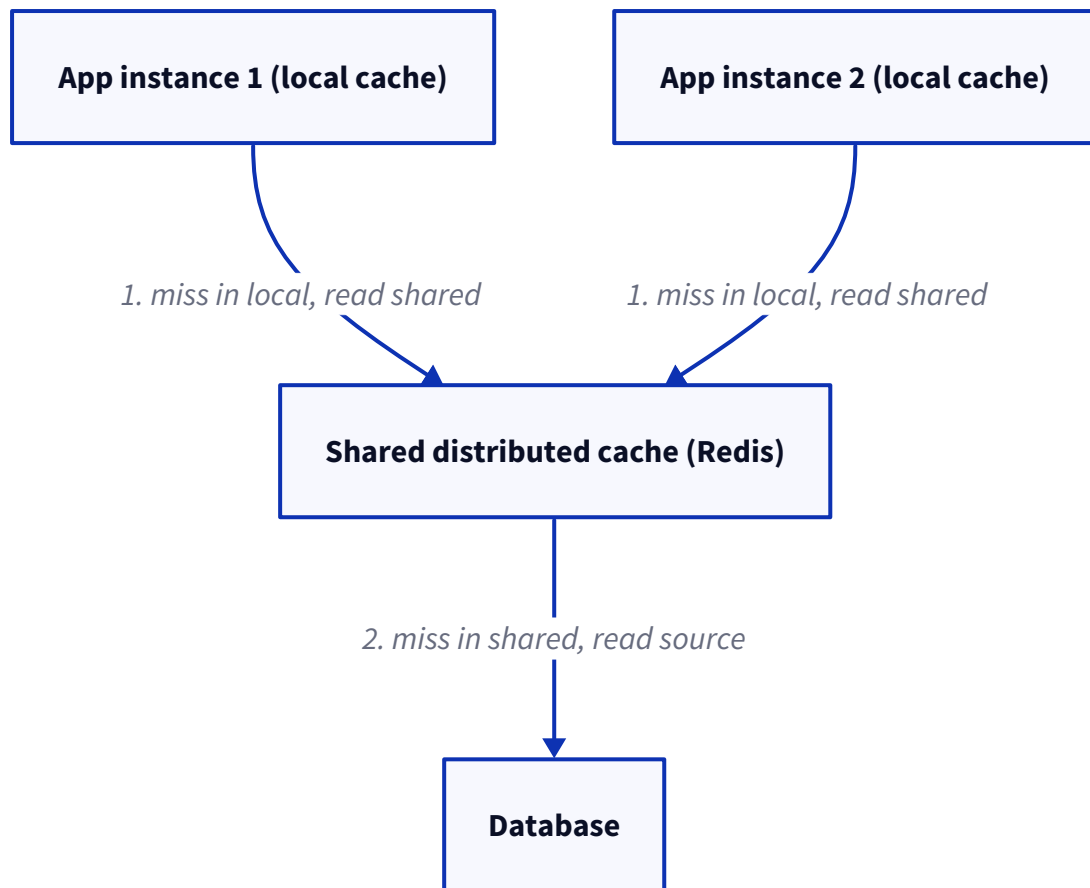
Memcached is a **simple, multi-threaded, in-memory key-value cache**. **Architecture:** multi-threaded (scales across cores), a slab allocator for memory, LRU eviction, no persistence, and no replication; values are opaque blobs and sharding is done client-side. Use it when you want a plain, fast, horizontally-scaled cache with no extra features.

14. Redis v/s Memcached.

Aspect	Redis	Memcached
Data types	Rich (lists, sets, sorted sets, ...)	Plain key-value
Threading	Mostly single-threaded	Multi-threaded
Persistence	Yes (RDB, AOF)	No
Replication / HA	Yes	No, client-side sharding only
Extras	Pub/sub, scripting, TTL, sorted sets	None
Best for	Rich, persistent, or HA cache	Simple large-scale key-value cache

15. How do multiple application instances share a cache?

By using a **distributed cache** (Redis, Memcached) that every instance talks to over the network, instead of each keeping its own in-process cache. A **local** cache is fastest but per-instance and instances can disagree; a **distributed** cache is shared and consistent at the cost of a network hop. Many systems use both: a small local cache in front of the shared one.



16. How are cache keys routed and replicated across nodes?

Keys are spread across nodes by **sharding**, usually **consistent hashing**, so adding or removing a node moves only a small slice of keys instead of rehashing everything. For availability each shard is **replicated** to one or more replicas, and a replica is promoted if the primary fails. Redis Cluster uses hash slots; Memcached leaves routing to the client.

17. Caching strategies.

Cache-aside (lazy loading). The app checks the cache and, on a miss, reads the source and populates the cache. The most common pattern; the cache holds only used data, but every miss hits the source.

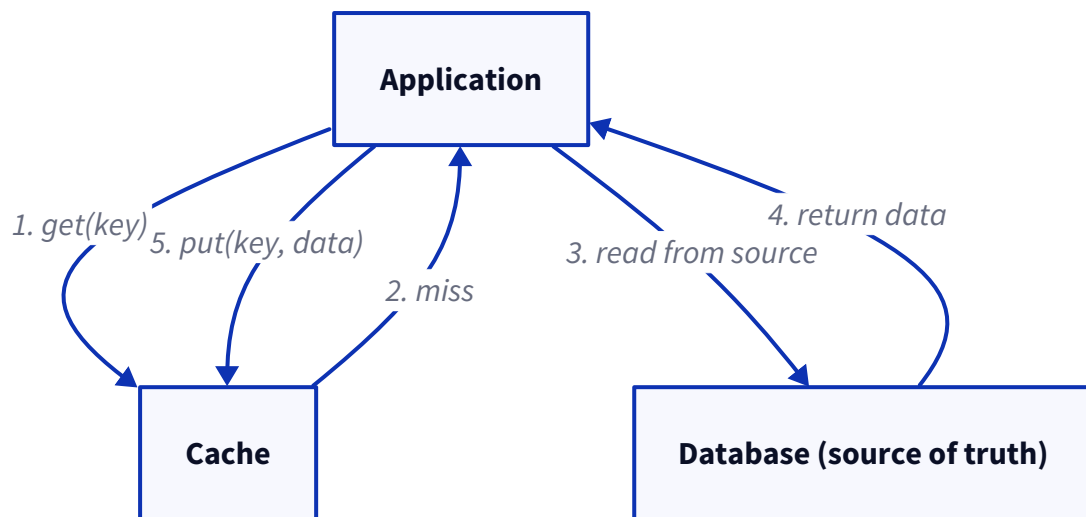
Read-through. The cache loads missing data itself; the app only asks the cache. Cleaner app code, at the cost of coupling to the cache library.

Write-through. Writes go to the cache and synchronously to the source. The cache stays in sync, but every write pays the source's latency.

Write-around. Writes go straight to the source and skip the cache; the cache fills on later reads. Avoids caching write-once data, but the first read after a write misses.

Write-behind (write-back). Writes go to the cache, which flushes to the source asynchronously. Fast writes, but data is lost if the cache dies before flushing.

Refresh-ahead. The cache refreshes hot entries before they expire, so users rarely see a slow miss. Great for predictable hot keys, wasteful for cold ones.



Strategy	Read path	Write path	Best for
Cache-aside	App checks cache, loads source on miss	App writes source, invalidates cache	General read-heavy data
Read-through	Cache loads on miss	(paired with write-through)	Clean app code
Write-through	From cache	Cache then source, synchronously	Cache must stay in sync
Write-around	From cache	Source only, skip cache	Write-once, read-rarely
Write-behind	From cache	Cache now, source later (async)	Write-heavy, latency-sensitive
Refresh-ahead	From cache	(any)	Predictable hot keys

18. What is cache consistency, and how do you keep the cache and source in sync?

Cache consistency is how closely the cache reflects the source of truth. You keep them in sync with **short TTLs** (bounded staleness), **write-through** (update both on write), or **invalidate-on-write** (delete the key on write so the next read reloads). Invalidating is usually safer than updating the cached value, because it avoids races between concurrent writers.

19. What is a cache stampede?

A **cache stampede** (thundering herd) is when a popular key expires and many requests miss at once, all hitting the source together and overloading it. Fixes: a **lock or single-flight** so only one request recomputes while the rest wait, **jittered TTLs** so keys do not expire together, **refresh-ahead**, and serving a slightly stale value while one refresh runs.

20. Why not cache everything?

Because most data is not read often enough to earn a cache slot. Caching cold data wastes memory, lowers the hit ratio, and adds staleness and invalidation work for no speed gain. Cache the hot, expensive, read-heavy slice and leave the rest to the source.

21. Database-first v/s cache-first.

Database-first (cache-aside): the database is the truth and the cache is an optimization on the side. **Cache-first**: read the cache and treat a miss as the exception. Database-first is the safe default; cache-first only fits data you can fully load and keep warm, and it needs a fallback for when the cache is cold or down.

22. What is CDN caching?

A **CDN** caches content on edge servers close to users, so a request is served from a nearby edge instead of the origin. Best for static assets (images, JS, CSS, video) and cacheable responses; it cuts latency and offloads the origin. Freshness is controlled with `Cache-Control` and cache purging.

23. Browser cache v/s CDN cache.

The **browser cache** is private to one user's device, so it helps that user's repeat visits. The **CDN cache** is shared across all users near an edge. Browser caching removes the round trip entirely; CDN caching shortens it for everyone. Both are driven by the same HTTP cache headers.

24. Where should cache live in microservices?

Prefer a **distributed cache owned by the service that owns the data**, so callers do not each cache it inconsistently. A small **local cache** in front is fine for very hot, rarely-changing data. Avoid caching another service's data across the boundary; ask that service, and let it cache its own.

25. Redis goes down. What happens?

If the code treats Redis as a **cache** (cache-aside), reads fall back to the database and keep working, just slower, until Redis returns; the risk is a stampede as everything misses at once. If the code treats Redis as the **only store** (sessions, primary data), those reads fail outright. This is why a cache should degrade to the source, not depend on the cache being up.

26. Users are seeing stale data. How would you debug it?

Trace the read path and find where the stale copy lives. Check the **TTL** and last write, whether the **write path invalidates** the key, and whether multiple layers (browser, CDN, app, Redis) each hold a copy. Common causes: a write that updated the database but not the cache, a CDN or browser holding an old response, or a race where a stale value was written back after an invalidation.

27. How would you cache user sessions for millions of users?

Store sessions in a **distributed cache** (Redis) keyed by session id, with a TTL matching the session lifetime, so any app instance can read them and the app tier stays stateless. Shard by session id and replicate for availability. Keep payloads small: store an id and load the rest on demand.

28. How would you cache search results?

Cache by a **normalized query key** (query plus filters, sort, and page) with a short TTL, because results change. Cache only common queries (the head), not every unique one, or the hit ratio collapses. Shorten the TTL or invalidate when the underlying data changes materially.

29. How would you cache permissions and roles?

Cache the resolved permissions per user with a **short TTL**, because access changes must take effect quickly, and **invalidate that user's entry** on any role or permission change so a revoked permission does not linger. Short TTL plus invalidate-on-change balances speed against the risk of stale access.

30. How does caching improve scalability?

By absorbing repeated reads so the source scales further. If 90% of reads hit the cache, the database serves a tenth of the traffic and supports many more users on the same hardware. The cache turns read scaling into a cheap memory problem instead of a database problem.

31. If Redis is much faster than a database, why not store everything only in Redis?

Because Redis is **memory-bound and not built as a system of record**. Everything in RAM is expensive at scale, its persistence is optional and weaker than a database's durability, and it lacks rich querying, joins, and transactional integrity over large datasets. Use Redis as a fast layer over a durable database, not as the durable store itself.

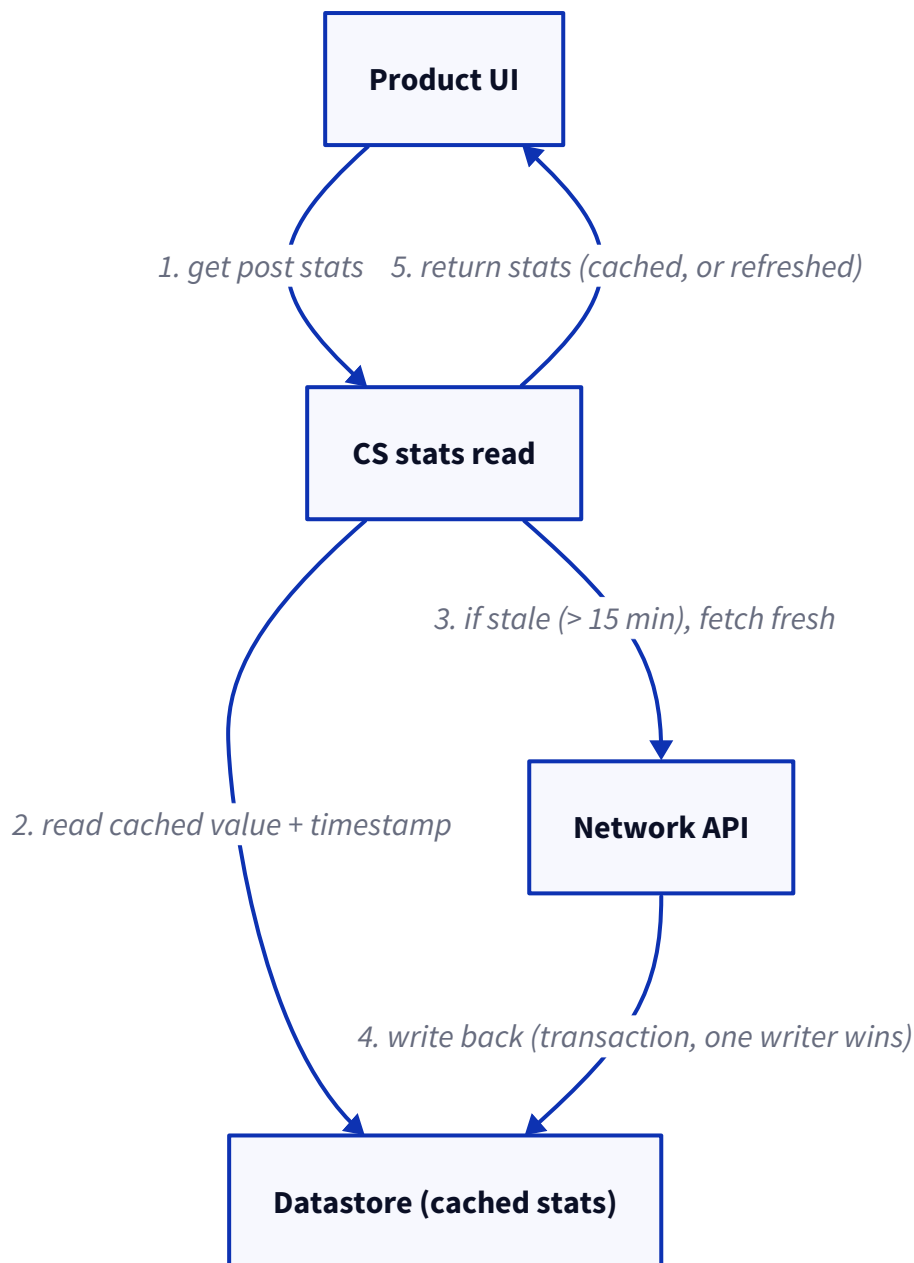
32. What principles decide whether a feature should cache, and which strategy fits?

Cache only when reads dominate writes, the data tolerates some staleness, and the hit ratio will be high enough to matter. Then pick the strategy by the workload: **cache-aside** for general read-heavy data, **write-through** when the cache must stay in sync, **write-behind** when write latency matters more than durability, **refresh-ahead** for predictable hot keys, and **write-around** for write-once data. Always design the cache to degrade to the source.

Vendasta-specific questions

1. Tell me about a caching bug or incident you dealt with: stale data, a stampede, a hot key, or a cache outage. What happened, and what did you change?

The one I would tell is about our **per-post stats cache**. We cache post stats in Datastore with a 15-minute freshness window, and when a read finds a stale value it goes and fetches fresh from the network API right there in the request, then writes it back. The trouble showed up when a lot of posts went stale at once, say a dashboard loading dozens of posts, or a burst after a quiet spell: each stale post fired its own synchronous network fetch on the request path. Latency spiked, and we ended up throwing a burst of calls at a rate-limited network API, which was getting close to a stampede.



What we changed was to stop doing the heavy refresh on the read path: serve the cached value straight away, and let a background job keep the hot posts warm, while the write-back transaction, where only one writer wins, collapses the concurrent refreshes down to one.

This is a representative version built on the real cache design; I would tell it with my own specifics in the room.

How did you detect it, and what was the blast radius?

We caught it from the latency percentiles on the stats endpoint climbing, and a spike in network-API calls and their rate-limit errors. The blast radius stayed inside the stats read path, so dashboards were slow and some refreshes failed, but posting and the rest of the product were fine, and nothing was lost because the cache just falls back to the stored value.

The fix, and how you stopped it recurring.

Serve the cached value, move the refresh to the background job, keep the single-writer write-back so concurrent refreshes collapse into one, and jitter the freshness so posts do not all go stale at the same moment.

The background job keeping hot posts warm is really what stops a read from ever triggering a synchronous fetch, so it does not come back.

Have you ever removed a cache because it caused more harm than it saved?

Yes, and the principle I would give is this: a cache with a low hit ratio, where the data changes faster than it is read, is just adding a network hop, memory, and staleness for almost nothing. When the hit rate stays low even after you bump the TTL, the honest move is to delete the cache and read the source directly.

2. How do you actually choose a TTL, and which metrics tell you the cache is healthy or hurting?

The way I pick a TTL is from two things: how stale the data is allowed to be, and how often it actually changes. Our own caches show the range, our account-group identifiers and the post-stat freshness use a 15-minute window because they barely change and a little lag is fine, model availability uses 30 seconds because it has to react fast, and a chat stream-resume entry uses a week because it is tied to the session, not to freshness. So I set the TTL to the largest staleness the feature can live with, then tighten it if users start noticing.

Hit rate, latency percentiles, eviction rate, origin load: which do you watch first?

I watch origin load and latency percentiles first, because those tell me whether the cache is actually doing its job of protecting the source and the user. Then hit rate, is the cache earning its keep, and eviction rate, is it too small and churning the hot keys out. On some of our caches we track hit and miss through statsd, and that is the signal I use for hit rate.

When does a low hit rate mean the cache is just overhead you should delete?

When the hit rate is low and the origin is not actually the bottleneck. A cache that mostly misses is just adding a hop, memory, and staleness while the source still does the work. And if bumping the TTL does not lift the hit rate, because the data changes too fast or the reads are too spread out, then I would delete it.

3. The product generates and serves images and short videos and pulls stock media. Where does CDN or edge caching fit, and how do you invalidate a changed asset?

Let me be straight about what we actually do: our generated images get uploaded to a public GCS bucket and served from the plain `storage.googleapis.com` URL, with no CDN, no cache-control header, and no signed URL in the app code. Stock media we mostly just pass through from the provider's own URL. So the edge caching we get today is whatever GCS, and any CDN sitting in front at the infra layer, gives us, not something the app is managing.

Here are the general techniques and where each one stops:

Cache-control headers and content-hashed immutable URLs.

The clean pattern is immutable, content-addressed URLs, a hash or a unique id in the path, plus a long `Cache-Control: immutable`, so the edge can cache forever and any change becomes a brand-new URL. We half-do this by accident, because every generation is written to a new random-UUID object, so a "changed" image is really a new URL, not an overwrite. What we are missing is the explicit long `Cache-Control`, which is a bucket or CDN setting, not app code.

Invalidating the edge v/s versioning the URL: which do you prefer and why?

I prefer versioning the URL. Purging an edge cache is slow, it is global, and it is easy to get wrong; whereas giving the changed asset a new immutable URL means the old one can sit there cached harmlessly and clients just fetch the new one. And our unique-object-per-generation already gives us that, so there is nothing to purge.

Signed URLs for private, per-business media: how does that interact with caching?

This is the tension: a signed, per-user, time-limited URL is basically uncacheable at a shared edge, because every URL is unique and it expires, so the edge cannot reuse it across users. So public media gets a cacheable public URL, which is what we use for generated images, and private media gets a signed URL and gives up shared-edge caching. We only sign the CSV export, not media; if we did have to make per-business media private, I would sign it and accept that we lose most edge caching, or put it behind an authenticated CDN.

4. The AI cost note mentions caching where prompts repeat. How would you cache AI generations safely, given every caption is grounded per business?

I will be honest up front: we do not actually cache AI generations at the application level today. Every caption or image generation calls the provider once it clears the IAM and rate-limit checks. The only prompt caching in play is provider-side prefix caching, where we keep the stable part of the prompt byte-identical so the model provider's own cache can serve it and save tokens within a conversation. It is not a Vendasta cache keyed by business. So what follows is how I would build one safely.

What is a safe cache key when the prompt embeds retrieved business context?

The key has to include everything that changes the output: the business, so the account group, the full retrieved context hashed in, the model, and the prompt-template version, not just the user's text. If the business context is part of the prompt, it has to be part of the key, otherwise two businesses with the same user text would collide.

INTERVIEW TRAP

Semantic caching is dangerous here. Two businesses can ask for a similar caption, so their prompts end up semantically close, and a "similar enough" match could serve **business A's on-brand voice to business B**. For per-business grounded output I would stay exact-match, or scope any semantic cache strictly inside one business.

How do you version or bust the cache when the prompt template changes?

I would put a template version right in the key, and bump it whenever the template or the grounding logic changes, so the old entries simply never get hit again, no purge needed. It is the same trick as immutable URLs, version the key and let the stale entries age out by TTL.

5. The product is white-label with many partners and businesses. How do you design cache keys and capacity so one hot partner does not evict everyone else, or leak data across tenants?

The good news is we already namespace our cache keys by tenant. The keys carry a service prefix and the account group id, for example `social-posts:ag_ids:{accountGroupID}`, and the shared Redis client puts a service-account prefix on top of that. Even our rate-limit keys are `ratelimit-{namespace}-{metric}-`

{consumerKey} , with the consumer key being the account group. So tenants are separated by the key itself, not by trusting the caller.

Key namespaces by partner and account group.

The rule is to prefix every key with the owning service and the tenant id, the account group, and the partner where it is relevant, so no two tenants can ever address the same entry. That is what our code does, and it is exactly what makes a cross-tenant collision impossible by construction.

A viral post creating a single hot key: how do you absorb it?

A viral post is just one hot key that everyone is reading. I would absorb it with the same read-through-plus-staleness pattern the stat cache uses: reads serve the cached value, and only one refresh runs per staleness window, because the write-back transaction lets a single writer win, so a million reads collapse into one refresh every 15 minutes. On top of that I would put a small local cache in front for the very hottest keys, and jitter the expiry so the hot keys do not all fall over at once.

THE RISK

If two tenants ever computed the **same key**, one would read the other's cached value, a cache-level cross-tenant leak, the same shape as the IDOR risk in the auth and BigQuery files. I rule it out by **always embedding the tenant id** (the account group, plus the service and service-account prefixes) in the key, and never keying a per-business entry on user text alone.